Test Plan

Team 7

2351441 许奕

2351039 王相

2350283 康凤轩

2352746 张洛梧

2350217 陈奕玮

Test Plan

1. Project Scope

1.1 Background

This test plan is written for the **independent application under test**, not for the AutoTestDesign tool itself.

The application under test is **MiniShop Checkout**, a compact e-commerce checkout prototype implemented in:

- `target_app/minishop_checkout/`

Its formal definition is documented in:

- `final_docs/12_target_application_definition_cn.md`

The final project uses `ARG-Test` as the AutoTestDesign tool to support the testing of `MiniShop Checkout`. The tool is responsible for:

- requirement ingestion and structuring
- risk-aware prioritization
- systematic black-box test generation
- export of auditable artifacts
- interactive review and revised-suite export

However, the **system under test in this plan** is `MiniShop Checkout`, not `ARG-Test`.

1.2 Overall Objectives

The objectives of this test plan are:

1. identify the major functional and non-functional test items of `MiniShop Checkout`
2. prioritize testing effort according to target-application risk
3. choose suitable techniques for each application component
4. execute one selected major module in detail with black-box and white-box evidence
5. use `ARG-Test` to make the design process more systematic, traceable, interactive, and reviewable
6. provide a consistent planning document that supports the risk report, detailed execution document, final PPT, and demo

1.3 Planning Assumptions

This plan uses the following assumptions:

- `MiniShop Checkout` is a compact prototype intended for a course project rather than for production deployment
- the application logic is stable enough for requirement-driven test design and local executable validation
- `ARG-Test` is used to accelerate analysis and traceability, but human review remains required before final acceptance
- only one selected major module needs full detailed execution depth for the assignment, while the remaining components may rely on structured design plus smoke-level executable support

1.4 In-Scope and Out-of-Scope Content

In scope:

- coupon and promotion behavior
- shipping fee calculation
- tax and order-total calculation
- payment-card validation
- pickup-station and recipient validation
- checkout orchestration across the above modules
- detailed executable testing for `coupon_discount_engine`
- traceable test artifacts and risk prioritization produced with `ARG-Test`
- course-scale NFR evidence for performance, usability, security, and maintainability

Out of scope:

- order-history and post-purchase account features
- refund execution and return approval workflows
- inventory synchronization and warehouse dispatch
- external payment gateway integration
- production deployment, high-concurrency load testing, or true networked payment processing

The project therefore combines **concrete application testing** for `MiniShop Checkout` with **tool-supported requirement-driven test design** from `ARG-Test`.

---

2. Test Items

2.1 Major Functional Features

Feature area	Main behavior under test	Main code path	Requirement basis
Promotion and pricing	coupon thresholds, coupon validity, member restriction, sale-item restriction, free-shipping effect	`target_app/minishop_checkout/promotion.py`, `reference_impl/coupon_discount_engine.py`	`coupon_discount_engine`
Shipping calculation	zone-based fee selection, weight tiers, express surcharge, free-shipping threshold	`target_app/minishop_checkout/shipping.py`	`shipping_fee_calculator`
Tax and total calculation	taxable base selection, country-specific rate, rounding, final total	`target_app/minishop_checkout/tax.py`	`order_total_tax_calculation`
Payment validation	cardholder name, card number, expiry window, CVV length, masked-number rejection	`target_app/minishop_checkout/payment.py`	`payment_card_expiry_and_cvv_validation`

Feature area	Main behavior under test	Main code path	Requirement basis
Pickup validation	pickup station ID, recipient phone, contactless pickup code, note length	`target_app/minishop_checkout/pickup.py`	`pickup_station_contact_validation`
Checkout orchestration	subtotal, promotion, shipping, tax, payment, and pickup outputs combined into one order preview	`target_app/minishop_checkout/checkout.py`	integration of the above requirement concerns
Cart/subtotal preparation	item price, quantity, item discount, and sale-item interactions before downstream pricing logic	`target_app/minishop_checkout/models.py`, `target_app/minishop_checkout/checkout.py`	subtotal assumptions used by promotion, shipping, and tax

2.2 Major Non-Functional Features

NFR area	Test interpretation for `MiniShop Checkout`	Planned evidence
Performance	checkout preview and validation should complete quickly for classroom-scale review usage	local NFR benchmark and smoke execution
Usability / understandability	order preview outputs and validation messages should be readable enough for review and debugging	`target_app/minishop_checkout/README.md`, demo paths, smoke scenarios
Security / data handling	malformed or masked payment input must be rejected and no secret material should appear in artifacts	`payment.py`, repository-local artifact scans
Maintainability and technology	components should stay small, responsibility-separated, and easy to test with `pytest`	`target_app/minishop_checkout/`, `tests/test_minishop_checkout_smoke.py`, repository test suite

2.3 Target Application Architecture and Main Components

The target application is a compact checkout-oriented system with the following layers and components.

Layer / component	Responsibility	Main code path
Input boundary	collect customer, cart, shipping, payment, and pickup input	conceptual application boundary
Cart and subtotal logic	aggregate item values and enforce item-level guards	`target_app/minishop_checkout/models.py`, `target_app/minishop_checkout/checkout.py`
Checkout service	coordinate subtotal, shipping, promotion, tax, and validation into one order preview	`target_app/minishop_checkout/checkout.py`
Promotion service	coupon validation and discount application	`target_app/minishop_checkout/promotion.py`, `reference_impl/coupon_discount_engine.py`
Shipping service	shipping fee calculation	`target_app/minishop_checkout/shipping.py`
Tax service	tax and total calculation	`target_app/minishop_checkout/tax.py`
Payment validation service	payment-card validation	`target_app/minishop_checkout/payment.py`
Pickup validation service	pickup contact validation	`target_app/minishop_checkout/pickup.py`

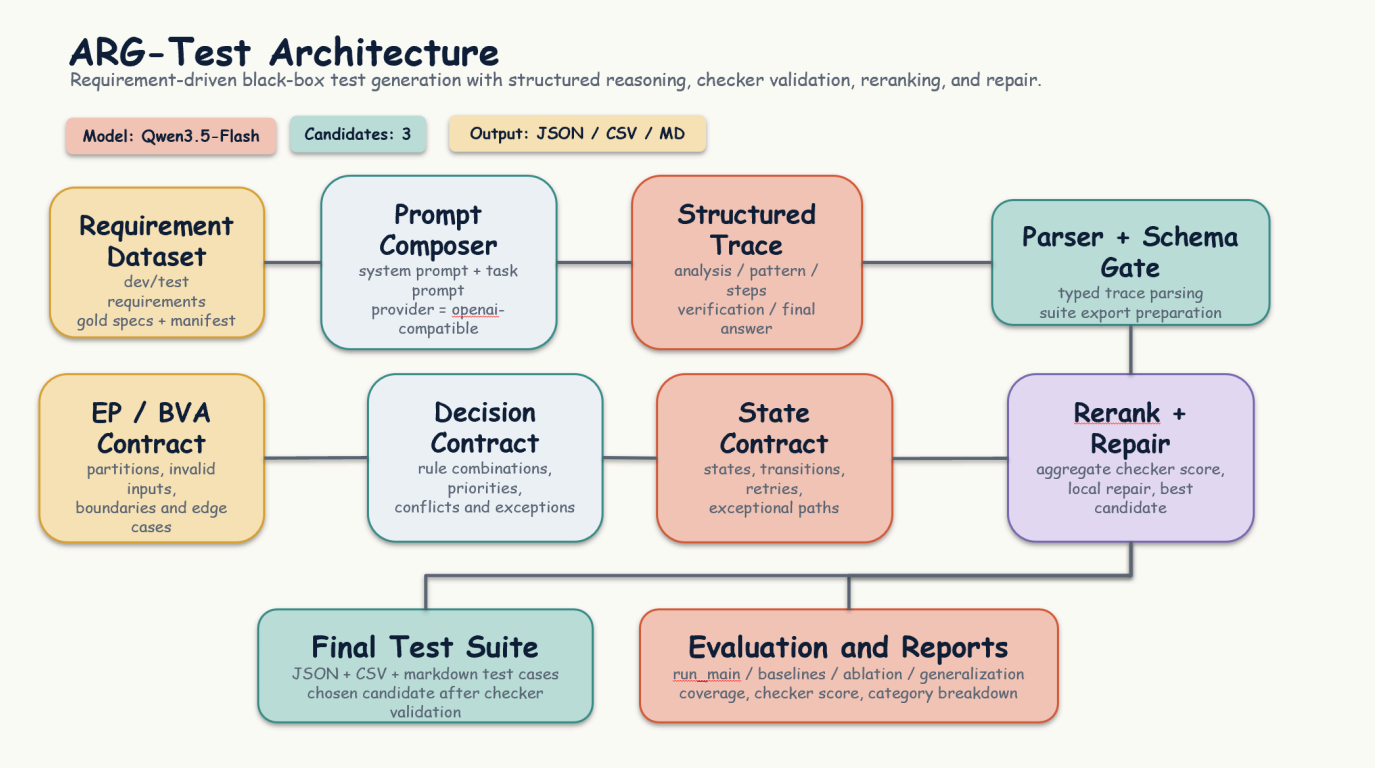
The main internal dependency chain is:

1. subtotal is prepared first
2. shipping quote depends on destination, method, weight, and subtotal
3. coupon logic consumes subtotal and shipping context
4. tax logic consumes discounted subtotal and shipping fee
5. payment and pickup validations execute in parallel with financial calculation
6. all component results are assembled into one `CheckoutSummary`

This dependency chain matters because it shapes both the test levels and the order of suite execution.

2.4 Supporting AutoTestDesign Tool Path

`MiniShop Checkout` is tested with `ARG-Test`, whose support architecture is shown below for traceability of the testing workflow.



The figure above explains how `MiniShop Checkout` requirements are ingested, structured, checked, reranked, repaired, and exported. It is included here as the **supporting test-tool path**, not as the architecture of the system under test.

---

3. Test Strategy and High-Level Suite Design

3.1 Planning Principles

The overall planning strategy follows five principles:

1. **risk first**
   - high-impact financial and validation areas receive the earliest and deepest coverage
2. **multiple techniques where necessary**
   - no single black-box technique is assumed to be sufficient for threshold-rich or interaction-heavy logic
3. **traceability over convenience**
   - every main suite must map back to application concerns and requirement obligations
4. **designer-in-the-loop review**
   - generated suites are reviewed and, when needed, revised before being treated as final evidence
5. **one deep executable anchor**
   - one major module is executed in detail so that the final project contains concrete script-level evidence, not only design-level artifacts

3.2 Planned Test Suites

Suite ID	Suite name	Main application focus	Main techniques	Primary goal	Main expected artifact
A	Promotion suite	coupon validity, threshold edges, premium restriction, sale-item conflict	`Decision Table + EP + BVA`	cover business-rule combinations and discount thresholds	reviewed generated suite + detailed module linkage
B	Shipping and tax suite	weight tiers, zone tiers, shipping threshold, taxable base, rounding	`BVA + Decision Table`	cover mathematical correctness and threshold edges	generated component-facing suites +

Suite ID	Suite name	Main application focus	Main techniques	Primary goal	Main expected artifact
					smoke calculations
C	Payment validation suite	required fields, format, length, range, masked-card rejection	`EP + BVA`	cover valid/invalid payment partitions	generated suites + explicit negative scenarios
D	Pickup validation suite	pickup station format, phone format, contactless pickup code rules	`EP + BVA`	cover required/optional and valid/invalid partitions	generated suites + smoke validation cases
E	Checkout orchestration suite	subtotal, promotion, shipping, tax, payment, and pickup consistency	scenario-based integration checks	verify correctness of overall preview assembly	smoke scenarios and summary assertions
F	Detailed executable module suite	`coupon_discount_engine`	`EP + BVA + Decision Table + white-box branch testing`	connect requirement-driven design to executable evidence	`pytest`, coverage, mutation evidence
G	NFR and traceability suite	performance, maintainability, artifact consistency, reviewability	benchmark, artifact verification, consistency checks	support final NFR and submission claims	NFR summary, test counts, validation artifacts

3.3 Technique Selection by Risk

Risk profile	Technique choice	Rationale
Promotion and coupon rules	`Decision Table + EP + BVA + selected white-box`	multiple interacting rules and explicit numeric thresholds
Shipping and tax calculation	`BVA + Decision Table + targeted arithmetic spot checks`	threshold edges, tier resolution, country-specific logic, and rounding
Payment and pickup validation	`EP + BVA`	best fit for valid/invalid partitions and inclusive boundaries
Checkout orchestration	scenario-based integration checks	needed to verify that individually correct component outputs stay consistent when combined
Selected executable module	`EP + BVA + Decision Table + white-box`	satisfies both black-box and white-box expectations for the detailed document

3.4 Designer-in-the-Loop Review Strategy

The assignment requires designer participation and interactive review. In this project, the testing workflow is therefore organized as:

1. select or enter a `MiniShop Checkout` requirement
2. let `ARG-Test` generate a structured trace and candidate suite
3. review risk signals, selected techniques, generated cases, and diagnostics
4. revise requirement wording or designer guidance before rerun when needed
5. directly edit generated test cases after generation when designer review identifies gaps
6. export the reviewed suite and use the detailed module for executable validation where applicable

The current practical review surfaces are:

- Direct Input

- CSV Batch
- State Model
- Formal Evidence

The current interactive review controls support:

- designer-selected technique emphasis
- explicit coverage-item review notes
- direct post-generation editing of generated test cases in the Web demo
- revised-suite export with preserved review metadata and refreshed checker/coverage diagnostics

3.5 Evidence Chain and Traceability

Plan item	Application concern	Verification activity	Main evidence
Promotion-rule coverage	coupon validity, threshold, and exclusivity logic	inspect generated obligations and detailed module evidence	`outputs/final_tests/`, `final_docs/execution_evidence/`
Shipping/tax correctness	wrong fee, taxable base, or rounding	review generated cases and smoke calculations	`target_app/minishop_checkout/`, smoke scenarios
Payment validation coverage	malformed payment data accepted or rejected incorrectly	inspect valid/invalid partitions and boundary cases	generated suites, `target_app/minishop_checkout/payment.py`
Pickup validation coverage	bad pickup contact data accepted	inspect EP/BVA cases and smoke checks	generated suites, `target_app/minishop_checkout/pickup.py`
Checkout integration consistency	incorrect total or dropped validation signals	execute end-to-end preview scenarios	`target_app/minishop_checkout/checkout.py`, `tests/test_minishop_checkout_smoke.py`
Detailed executable module	one major module must be executed in detail	run `pytest`, `coverage.py`, and mutation evidence for `coupon_discount_engine`	`tests/`, `reference_impl/`, `final_docs/execution_evidence/`
NFR and maintainability evidence	local speed, usability, modularity, regression strength	run NFR checks and repository regression suite	`final_docs/execution_evidence/nfr_validation_summary.*`, `tests/`

3.6 Test Data Sources

The plan uses several sources of data:

Data source	Use in this plan
`data/requirements/`	requirement text for tool-supported generation and review
`target_app/minishop_checkout/` logic	executable basis for application-facing suites
`tests/test_minishop_checkout_smoke.py` scenarios	end-to-end application smoke coverage
`reference_impl/coupon_discount_engine.py`	selected major module under detailed execution
`tests/test_coupon_discount_engine_blackbox.py` and `tests/test_coupon_discount_engine_whitebox.py`	executable detailed test scripts
`data/gold_specs/`	obligation-based evaluation for tool outputs

4. Test Levels, Environments, and Goals

4.1 Test Levels

Test level	Goal	Typical output
Requirement-level design	verify that `MiniShop Checkout` requirements are translated into correct coverage items and techniques	structured traces, risk scores, generated suites
Component-level execution	verify shipping, tax, payment, pickup, and orchestration behavior with executable smoke scenarios	smoke outputs, component assertions
Module-level detailed execution	verify one selected major application module with full executable evidence	`pytest` results, coverage, mutation evidence
Acceptance / submission level	verify that all deliverables consistently describe `MiniShop Checkout` and its selected module	final documents, evidence package, PPT wording

4.2 Test Environment

Environment item	Planned setting
Language/runtime	Python
Main test execution framework	`pytest`
Coverage collection	`coverage.py --branch`
Tool-assisted design path	`ARG-Test` CLI and Web demo
Default stable demo provider	`mock`
Output roots	repository-local tracked outputs and `.local_runs/` for replayable sessions

4.3 Entry Criteria

The main entry criteria for this plan are:

1. the target application scope is frozen in `final_docs/12_target_application_definition_cn.md`
2. component code paths are identifiable and runnable
3. the risk report exists and identifies the main risk clusters
4. the selected major module for detailed execution is fixed
5. the team can run `pytest` and the AutoTestDesign workflow locally

5. Schedule and Checklist

The schedule below is organized around **testing** `MiniShop Checkout`, not around generic repository maintenance.

5.1 Phase Schedule

Phase	Main activity	Deliverable / checkpoint
Phase 1	freeze the target-application definition and module map	`12_target_application_definition_cn.md`, `target_app/minishop_checkout/`
Phase 2	perform application risk analysis and prioritize suites	risk register and suite priorities
Phase 3	generate and review application-facing suites with `ARG-Test`	reviewed black-box suites for the main components
Phase 4	perform detailed executable testing for `coupon_discount_engine`	black-box tests, white-box tests, coverage, mutation evidence
Phase 5	run target-application smoke checks and verify traceability	smoke scenarios, consistent code/document references
Phase 6	consolidate documents and submission artifacts	PDF documents, PPT/PDF, demo package, zipped scripts

5.2 Pre-Submission Checklist

1. the risk report, test plan, and detailed execution document all describe `MiniShop Checkout`
2. `coupon_discount_engine` is consistently described as a selected module of `MiniShop Checkout`
3. the main risk areas have assigned suites and stated techniques
4. the selected executable module has black-box and white-box evidence
5. the final package does not confuse `ARG-Test` with the application under test
6. the demo and PPT wording match the formal documents
7. the latest repository evidence numbers are synchronized across report, PPT, and execution summaries

---

6. Organization Structure and Responsibilities

6.1 Member Responsibilities

The team structure below describes responsibilities for testing `MiniShop Checkout` with `ARG-Test`.

Member	Student ID	Main responsibility in target-application testing
Yi Xu	2351441	overall integration, application-scope consistency, tool-run control, final merge
Luowu Zhang	2352746	document integration, PPT, demo assets, wording consistency across application-facing deliverables
Xiang Wang	2351039	requirement assets, target-application scenario maintenance, traceability support
Fengxuan Kang	2350283	selected executable module support, `pytest` execution support, smoke checks
Yiwei Chen	2350217	evaluation interpretation, reproducibility support, verification materials

6.2 Responsibility Flow

- freeze application scope first
- align risk and suite priorities next
- review generated suites and adjust wording if needed
- execute the selected detailed module

- synchronize documents and evidence to the same target-application wording

6.3 Lightweight RACI View

Activity	Responsible	Accountable	Consulted	Informed
target-application definition	Yi Xu, Xiang Wang	Yi Xu	all members	all members
risk report	Yi Xu, Luowu Zhang	Yi Xu	Xiang Wang	all members
suite generation and review	Xiang Wang, Yiwei Chen	Yi Xu	all members	all members
detailed execution	Fengxuan Kang	Yi Xu	Yiwei Chen	all members
PPT and demo integration	Luowu Zhang	Yi Xu	all members	all members

---

7. Chosen Testing Framework and Rationale

7.1 Main Execution Framework

`pytest` is the chosen execution framework for the executable parts of `MiniShop Checkout`, especially the selected detailed module `coupon_discount_engine`.

Why `pytest`:

- the target application prototype is implemented in Python
- black-box, white-box, and smoke scenarios can all be expressed directly as assertions
- the framework integrates naturally with `coverage.py`
- it supports stable local reruns and selective execution for the final evidence package
- it keeps the detailed document reproducible and inspectable by course reviewers

7.2 Supporting Tools

Tool	Role in testing `MiniShop Checkout`
`ARG-Test`	generate structured requirement-driven suites and risk ordering
`pytest`	execute detailed black-box, white-box, and smoke tests
`coverage.py`	collect statement and branch coverage for the selected detailed module
mutation demo scripts	show defect-detection usefulness of the selected detailed module suite
FastAPI demo + TestClient	demonstrate and validate the interactive review path of the tool

This means `ARG-Test` is the **design-support tool**, while `pytest` is the **execution framework** used on the target application.

---

8. Cost Estimation

This cost estimate is for **using the developed AutoTestDesign tool to test** `MiniShop Checkout`. It does **not** count the original R&D cost of building `ARG-Test` itself.

8.1 Estimation Assumptions

The estimate assumes:

- one compact target application
- one deeply executed selected module
- the rest of the application covered by suites plus smoke-level executable support
- the team already has the tool available

8.2 Estimated Workload with AutoTestDesign

Work item	Estimated workload
normalize `MiniShop Checkout` requirement sources and component map	`0.5 to 1.0` person-day
risk analysis and suite prioritization	`0.5 to 1.0` person-day
automated generation plus human review of component-facing suites	`1.5 to 2.0` person-days
selected module executable testing with `pytest` and `coverage.py`	`1.5 to 2.0` person-days
result consolidation and traceability packaging	`0.5 to 1.0` person-day

Estimated total with the tool:

- `4.5 to 7.0 person-days`

8.3 Manual-Testing Baseline Estimate

If the same target-application scope were tested manually without `ARG-Test`, the expected workload would increase because the team would have to:

- manually decompose each requirement into coverage items
- manually select techniques for each component
- manually keep risk priority and traceability aligned
- manually format and maintain export artifacts
- manually repeat the review packaging work after each iteration

Estimated manual workload:

- `7.5 to 10.0 person-days`

8.4 Interpretation

The main cost reduction from `ARG-Test` comes from:

- faster requirement decomposition
- faster first-pass suite generation
- faster risk-based ordering
- lower traceability and artifact-formatting overhead

Human review is still required for:

- checking semantic correctness of generated cases
- confirming high-risk rule combinations
- selecting and executing the detailed module
- packaging final evidence for submission and Q&A

The realistic interpretation is therefore:

- `ARG-Test` reduces planning and formatting cost
- it does not eliminate human responsibility for final correctness

---

9. Defect Handling and Reporting Strategy

This course project does not use a full industrial defect tracker, but the testing workflow still needs a consistent interpretation path.

Defects found during target-application testing are handled in three categories:

Defect category	Example	Planned action
Requirement-level gap	missing invalid partition, missing threshold obligation	revise suite through `ARG-Test` review flow and re-export

Defect category	Example	Planned action
Application logic defect	wrong fee, wrong discount, wrong validation behavior	capture failing scenario and update executable test or smoke scenario
Evidence-consistency defect	report, PPT, and execution summary use different numbers or wording	correct the source document and resynchronize derived materials

The main reporting artifacts are:

- reviewed generated suites
- smoke-test evidence
- `pytest` results
- coverage reports
- mutation evidence
- final documents and PPT wording

---

10. Exit Criteria

This test plan reaches its delivery condition when all of the following are true:

1. `MiniShop Checkout` has a fixed scope and component definition
2. the risk analysis report, test plan, and detailed execution document all point to the same application under test
3. high-risk application areas have mapped suites and selected techniques
4. `coupon_discount_engine` has complete black-box and white-box execution evidence
5. the target-application smoke scenarios are locatable and explainable
6. the final package does not confuse `ARG-Test` with `MiniShop Checkout`

For the selected module, the concrete execution acceptance target is:

- `15` module-focused tests passed
- `100%` statement coverage
- `100%` branch coverage
- `4 / 4` mutants killed

For the repository as a whole, the supporting regression expectation is:

- current automated regression suite remains passing at repository level
- key evidence counts remain synchronized across final materials

---

11. Conclusion

This test plan is centered on **MiniShop Checkout** and uses `ARG-Test` as the testing tool that supports requirement-driven design, prioritization, and traceability.

Its most important conceptual clarification is:

- `MiniShop Checkout` is the application under test
- `ARG-Test` is the tool used to test it
- `coupon_discount_engine` is the selected major module of `MiniShop Checkout` used for detailed execution

The plan is intentionally risk-based rather than evenly distributed. It gives the most testing weight to:

- promotion logic
- tax and order-total correctness
- payment validation
- checkout orchestration consistency

With that separation and prioritization fixed, the plan aligns with the assignment requirement that the test plan must describe how the team tests the **target application**, not the AutoTestDesign tool itself.